

Pythonia

Manual de estudio: tronco común

Maestro

Nivel 4: Experto

Entender Python por dentro y diseñar sistemas que duran.

4

10 lecciones | 12 semanas | 1 proyecto integrador

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Dominar el modelo de datos, la metaprogramación, los internals de CPython, la optimización, la arquitectura y la calidad de software a nivel profesional.

Cada lección tiene la misma estructura: explicación breve, un ejemplo para escribir y ejecutar, la salida real que produce, consejos, un ejercicio y una autoevaluación (respuestas al final). Desde este nivel, cada lección tiene además un **ejercicio con lente** de tu ruta de especialización, reunidos en el anexo final.

Además, cada lección tiene tres apartados de profundización (**el concepto a fondo**, **el ejemplo línea por línea** y **errores frecuentes**) y una **práctica evaluable**: un ejercicio que escribes en la app y se corrige automáticamente con pruebas. Cada práctica es una pieza del proyecto del nivel, que se va armando solo a medida que avanzas.

Método: lee en la app, escribe el ejemplo a mano, predice la salida, ejecútalo y anota el resultado en una ficha de **Registro de ejecución** del Cuadernillo-Diario. Cuando apruebes la práctica, regístrala en la **Bitácora de prácticas**: intentos, pistas usadas, el error que más te costó y qué aprendiste.

Tu plan

Semana	Temas	Misión de la semana
1	1. El modelo de datos a fondo	Crea una clase Matriz que soporte +, *, ==, len(), indexación m[i][j] e iteración por filas.
2	2. Descriptores y properties	Crea descriptores EnRango(min, max) y NoVacio y úsalos en una clase Empleado.
3	3. Metaprogramación	Construye un sistema de comandos donde cada subclase de Comando se registre sola y un CLI la ejecute por nombre.
4	4. Internals de CPython	Usa dis para comparar el bytecode de un for con append vs. una comprensión, y explica la diferencia.
5	5. Profiling y optimización	Perfila uno de tus scripts con cProfile, identifica las 3 funciones más lentas y mejora al menos una.
6	6. Concurrencia avanzada	Crea un crawler que visite 50 URLs simuladas con máximo 5 conexiones simultáneas y manejo de fallos parciales.
7	7. Patrones de diseño pythónicos	Refactoriza tu monitor de precios para que las fuentes de datos y los notificadores sean intercambiables vía Protocol.
8	8. Testing avanzado	Alcanza 90 % de cobertura en tu monitor de precios usando fixtures, parametrize y mocks.
9	9. Rendimiento nativo	Calcula estadísticas de un millón de números con bucles puros y con NumPy, y compara tiempos con timeit.
10	10. Arquitectura y calidad	Configura ruff, mypy, pre-commit y un workflow de CI para tu proyecto del nivel 3.
11	Proyecto integrador: Framework de automatización con plugins	Construir, probar y documentar el proyecto

Semana	Temas	Misión de la semana
12	Proyecto integrador: Framework de automatización con plugins	Construir, probar y documentar el proyecto

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

1 El modelo de datos a fondo

Todo en Python es un objeto, y casi toda la sintaxis se traduce en métodos especiales: $a + b$ es `a.__add__(b)`, `len(x)` es `x.__len__()`, `x[i]` es `x.__getitem__(i)`.

Implementar protocolos (`__iter__`, `__getitem__`, `__call__`, `__enter__`, `__hash__`) hace que tus objetos se comporten como nativos.

`__slots__` elimina el `__dict__` por instancia: ahorra memoria cuando creas millones de objetos y restringe qué atributos existen.

El concepto a fondo

Todo en Python es un objeto y casi toda la sintaxis se traduce en llamadas a métodos especiales (dunder): $a + b$ llama a `a.__add__(b)`, `len(x)` a `x.__len__()`, `x[i]` a `x.__getitem__(i)`, `for` usa `__iter__` y `obj()` usa `__call__`.

Implementar estos métodos hace que tus clases se comporten como tipos nativos: se pueden sumar, medir, indexar, recorrer y llamar. Es lo que permite que pandas, NumPy o pathlib se sientan parte del lenguaje.

Si un método binario no sabe operar con el tipo recibido, debe devolver `NotImplemented` (no lanzar un error): Python probará entonces la operación reflejada del otro objeto (`__radd__`) y, si nadie puede, lanzará `TypeError` solo.

`__repr__` debe mostrar el objeto sin ambigüedad (idealmente como se construiría) y `__str__` una versión amigable. `__bool__` o, en su defecto, `__len__` deciden si el objeto es verdadero: un contenedor vacío es falso.

Ejemplo

```
1 class Vector:
2     __slots__ = ("x", "y")
3     def __init__(self, x, y):
4         self.x, self.y = x, y
5     def __add__(self, otro):
6         return Vector(self.x + otro.x, self.y + otro.y)
7     def __mul__(self, k):
8         return Vector(self.x * k, self.y * k)
9     __rmul__ = __mul__
10    def __abs__(self):
11        return (self.x ** 2 + self.y ** 2) ** 0.5
12    def __repr__(self):
13        return f"Vector({self.x}, {self.y})"
14
15    v = Vector(3, 4)
16    print(v + Vector(1, 1), 2 * v, abs(v))
```

Salida real

```
Vector(4, 5) Vector(6, 8) 5.0
```

El ejemplo, línea por línea

1. `__slots__` fija los atributos permitidos y ahorra memoria.
2. `__add__` y `__mul__` definen $+$ y $*$; `__rmul__ = __mul__` permite $2 * v$.
3. `__abs__` responde a `abs(v)` con la longitud del vector.

4. `__repr__` define cómo se ve al imprimirlo.**Consejos**

- Si defines `__eq__`, define también `__hash__` (o el objeto deja de ser hashable).
- Devuelve `NotImplemented` (no lanzar error) cuando un operador no soporta el tipo recibido.

Ejercicio

Crea una clase `Matriz` que soporte `+`, `*`, `==`, `len()`, indexación `m[i][j]` e iteración por filas.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>TypeError: unsupported operand type(s) for +</code>	Tu <code>__add__</code> devolvió <code>NotImplemented</code> y el otro objeto tampoco supo sumar: es el comportamiento correcto ante tipos incompatibles.
<code>RecursionError en __repr__</code>	Dentro de <code>__repr__</code> usaste <code>repr(self)</code> o una f-string con <code>{self}</code> .

Práctica evaluable

Un Pipeline que se comporta como un tipo nativo · Pieza del proyecto: **Pipeline (modelo de datos)**

Crea `Pipeline(*pasos)`, una secuencia de funciones que se aplican en orden a un dato:

- `len(p)`, `for paso in p` y `p[0]` funcionan como en una lista; `p[1:]` devuelve un nuevo `Pipeline`.
- `p1 + p2` devuelve un `Pipeline` nuevo con los pasos de ambos (sin modificarlos). Sumar algo que no es `Pipeline` debe terminar en `TypeError`.
- `p(dato)` aplica todos los pasos en orden y devuelve el resultado.
- `repr(p)` es `"Pipeline(limpiar → mayus)"` con los nombres de las funciones, o `"Pipeline()"` si está vacío.
- Un `Pipeline` vacío es falso en un `if`.

Plantilla para empezar

```

1 class Pipeline:
2     """Pipeline(paso1, paso2)(dato) == paso2(paso1(dato))"""
3
4     def __init__(self, *pasos):
5         self.pasos = list(pasos)
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué logra `__slots__`?

- Hace la clase inmutable
- Hace la clase abstracta
- Evita el `__dict__` por instancia y ahorra memoria
- Acelera los métodos

2 Descriptores y properties

Un descriptor es un objeto con `__get__`, `__set__` o `__delete__` que controla el acceso a un atributo de otra clase.

`property`, `classmethod`, `staticmethod` y los métodos mismos están implementados como descriptores.

Son ideales para validación reutilizable: escribes la regla una vez y la aplicas a muchos atributos.

El concepto a fondo

Un descriptor es un objeto, definido como atributo de clase, que controla el acceso a ese atributo en las instancias mediante `__get__`, `__set__` y `__delete__`. Así funcionan por dentro `property`, los métodos y los campos de los ORM como SQLAlchemy.

`__set_name__(self, dueño, nombre)` se llama al crear la clase dueña y le dice al descriptor con qué nombre fue asignado. Con eso puede guardar el valor real en un atributo privado de la instancia, como `_reintentos`.

`__get__(self, obj, tipo)` recibe `obj=None` cuando se accede desde la clase (`Config.reintentos`); por convención devuelve el propio descriptor en ese caso. Cuando se accede desde una instancia, devuelve el valor guardado.

`@property` es un descriptor listo para un solo atributo. Cuando la misma regla se repite en muchos atributos o clases (valores positivos, rangos, tipos), un descriptor propio evita copiar la validación en cada setter.

Ejemplo

```

1  class Positivo:
2      def __set_name__(self, owner, name):
3          self.privado = "_" + name
4      def __get__(self, obj, tipo=None):
5          return getattr(obj, self.privado)
6      def __set__(self, obj, valor):
7          if valor <= 0:
8              raise ValueError(f"{self.privado[1:]} debe ser > 0")
9              setattr(obj, self.privado, valor)
10
11 class Pedido:
12     cantidad = Positivo()
13     precio = Positivo()
14     def __init__(self, cantidad, precio):
15         self.cantidad, self.precio = cantidad, precio
16
17 p = Pedido(3, 12.5)
18 print(p.cantidad * p.precio)
19 try:
20     p.cantidad = -1
21 except ValueError as e:
22     print(e)

```

Salida real

```

37.5
cantidad debe ser > 0

```

El ejemplo, línea por línea

1. `__set_name__` guarda el nombre privado (por ejemplo `_cantidad`).

2. `__get__` lee el valor desde la instancia.
3. `__set__` valida antes de guardar y lanza `ValueError` si no cumple.
4. La validación corre también dentro de `__init__`, porque asignar pasa por el descriptor.

Consejos

- `__set_name__` (Python 3.6+) le dice al descriptor con qué nombre fue asignado.
- Para un solo atributo, `@property` es más simple; los descriptors brillan al reutilizar.

Ejercicio

Crea descriptors `EnRango(min, max)` y `NoVacio` y úsalos en una clase `Empleado`.

Errores frecuentes

Error frecuente	Cómo resolverlo
RecursionError en <code>__set__</code>	Guardaste con <code>setattr(obj, nombre_publico, ...)</code> , que vuelve a llamar al descriptor. Usa el nombre privado.
Todas las instancias comparten el valor	Guardaste el valor en el descriptor (<code>self.valor</code>), que es uno solo por clase. Guárdalo en <code>obj</code> .

Práctica evaluable

Configuración validada con descriptors · Pieza del proyecto: **Config con descriptors**

- Crea el descriptor `Rango(minimo, maximo)`. Al asignar, si el valor no es un número (`int` o `float`, excluyendo `bool`) lanza `TypeError("reintentos debe ser un número")`; si está fuera del rango, `ValueError("reintentos debe estar entre 0 y 10")`, usando el nombre real del atributo. Accedido desde la clase, devuelve el propio descriptor.
- Crea `Config` con `reintentos = Rango(0, 10)` y `timeout = Rango(1, 60)`, y un `__init__(self, reintentos=3, timeout=10)`.

Plantilla para empezar

```

1 class Rango:
2     """Descriptor que valida que un número esté en [minimo, maximo]."""
3
4     def __init__(self, minimo, maximo):
5         self.minimo, self.maximo = minimo, maximo
6
7
8 class Config:
9     reintentos = Rango(0, 10)
10    timeout = Rango(1, 60)

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿property es un descriptor?

- a) Solo con setter
- b) No
- c) Solo en Python 2
- d) Sí

3 Metaprogramación

Las clases también son objetos, creados por una metaclass. La metaclass por defecto es type.

`__init_subclass__` permite reaccionar cada vez que se crea una subclase: registro de plugins, validación de contratos, configuración automática. Cubre la mayoría de los casos sin metaclasses.

Las metaclasses propias (`class Meta(type)`) son la herramienta más potente y más peligrosa: úsalas solo cuando nada más sirve (ORMs, frameworks).

El concepto a fondo

Las clases también son objetos, creados por una metaclass. La metaclass por defecto es type: `type(Plugin)` es type, y hasta type es instancia de sí misma. Metaprogramar es escribir código que crea o modifica código, como clases y funciones.

Para la mayoría de los casos no hace falta una metaclass: `__init_subclass__` es un gancho que corre cada vez que se define una subclase. Es ideal para registrar plugins automáticamente, validar que las subclases definan ciertos métodos o configurarlas.

Las subclases pueden recibir argumentos con nombre en la línea de la clase: `class Mayus(Plugin, nombre="mayus")`. Llegan como parámetros a `__init_subclass__`, que debe llamar a `super().__init_subclass__(**kw)` para cooperar con otras clases.

Otras herramientas de metaprogramación: decoradores de clase, `getattr` / `setattr` dinámicos, `type(nombre, bases, dict)` para crear clases al vuelo y `__getattr__` para atributos calculados. Úsalas con medida: el código mágico es difícil de leer.

Ejemplo

```

1 class Plugin:
2     registro = {}
3     def __init_subclass__(cls, clave, **kw):
4         super().__init_subclass__(**kw)
5         Plugin.registro[clave] = cls
6
7 class ExportarCSV(Plugin, clave="csv"): ...
8 class ExportarPDF(Plugin, clave="pdf"): ...
9
10 print(list(Plugin.registro))
11 print(Plugin.registro["pdf"].__name__)
12 print(type(Plugin), type(type))

```

Salida real

```
['csv', 'pdf']
ExportarPDF
<class 'type'> <class 'type'>
```

El ejemplo, línea por línea

1. `__init_subclass__` recibe la clave de cada subclase y la guarda en el registro.
2. Al definir `ExportarCSV` y `ExportarPDF`, se registran solas.
3. `type(Plugin)` es `type` y `type(type)` también.

Consejos

- Regla de Tim Peters: si dudas de si necesitas una metaclass, no la necesitas.
- `getattr`, `setattr` y `type(nombre, bases, dict)` permiten crear clases dinámicamente.

Ejercicio

Construye un sistema de comandos donde cada subclase de `Comando` se registre sola y un CLI la ejecute por nombre.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>TypeError: __init_subclass__() missing 1 required argument</code>	Definiste una subclase sin el argumento <code>nombre=</code> en la línea de la clase.
Plugins que no aparecen en el registro	El módulo que los define nunca se importó: el registro ocurre al definir la clase.

Práctica evaluable

Plugins que se registran solos · Pieza del proyecto: Registro automático de plugins

- Crea la clase base `Plugin` con un diccionario de clase `registro`. Usa `__init_subclass__(cls, nombre, **kw)` para que cada subclase se registre sola con su nombre, guarde `cls.nombre` y, si el nombre ya existe, lance `ValueError`.
- `Plugin.crear(nombre)` (método de clase) devuelve una instancia del plugin; si no existe, lanza `KeyError` con el mensaje "Plugin desconocido: xxx".
- `ejecutar(dato)` en la base lanza `NotImplementedError`.
- Define dos plugins: `Mayusculas` (`nombre="mayus"`, pasa a mayúsculas) y `Recortar` (`nombre="recortar"`, quita espacios de los bordes).

Plantilla para empezar

```
1 class Plugin:
2     """Base de los plugins: class MiPlugin(Plugin, nombre="mi_plugin")"""
3     registro = {}
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuál es la metaclass por defecto en Python?

- a) class
- b) type
- c) Meta
- d) object

4 Internals de CPython

CPython compila tu código a bytecode y lo ejecuta en una máquina virtual de pila. El módulo `dis` te deja verlo.

La memoria se gestiona con conteo de referencias más un recolector de ciclos (`gc`). `is` compara identidad (mismo objeto); `==` compara valor.

El GIL permite que solo un hilo ejecute bytecode a la vez. Python 3.13 introdujo una versión experimental sin GIL (free-threaded) y un compilador JIT experimental.

El concepto a fondo

CPython compila tu código a bytecode y lo ejecuta en una máquina virtual de pila. `dis.dis(funcion)` muestra esas instrucciones: verlas ayuda a entender qué cuesta cada línea.

Las variables son nombres que apuntan a objetos. `a = b` no copia: hace que dos nombres apunten al mismo objeto. `==` compara valores; `is` compara identidad (si son el mismo objeto). Por eso modificar una lista desde un nombre se ve desde el otro.

Los valores por defecto se evalúan una sola vez, al definir la función. Un `def f(lista=[])` comparte la misma lista entre todas las llamadas: es uno de los errores más famosos de Python. La solución es `lista=None` y crearla adentro.

Una copia superficial (`dict(x)`, `lista[:]`, `copy.copy`) duplica solo el primer nivel: las listas internas siguen compartidas. `copy.deepcopy` duplica todo. Las clausuras guardan variables del ámbito exterior en celdas; `nonlocal` permite modificarlas.

Ejemplo

```

1  import dis, sys
2
3  def doble(x):
4      return x * 2
5
6  dis.dis(doble)
7
8  a = [1, 2]
9  print("referencias:", sys.getrefcount(a) - 1)
10 print([1, 2] == [1, 2], [1, 2] is [1, 2])

```

Salida real (el bytecode varía según la versión de Python)

```

3          0 RESUME               0

4          2 LOAD_FAST            0 (x)
          4 LOAD_CONST            1 (2)
          6 BINARY_OP             5 (*)
         10 RETURN_VALUE

referencias: 1
True False

```

El ejemplo, línea por línea

1. `dis.dis(doble)` muestra las instrucciones de la función.
2. `sys.getrefcount` cuenta cuántas referencias apuntan a la lista.
3. Dos listas iguales son `==` pero no son el mismo objeto (`is` da `False`).

Consejos

- El bytecode cambia entre versiones de Python: la salida de `dis` no es portable.
- Compara con `None` usando `is None`, nunca `== None`.

Ejercicio

Usa `dis` para comparar el bytecode de un `for` con `append` vs. una comprensión, y explica la diferencia.

Errores frecuentes

Error frecuente	Cómo resolverlo
La lista por defecto acumula valores entre llamadas	Usa <code>def f(lista=None):</code> y adentro <code>if lista is None: lista = []</code> .
Modifiqué la copia y cambió el original	Hiciste una copia superficial de una estructura anidada. Usa <code>copy.deepcopy</code> .

Práctica evaluable

Referencias, valores por defecto y clausuras • Pieza del proyecto: Utilidades del motor

Este código tiene dos errores clásicos de referencias y una función por completar:

- `agregar_paso(paso, pasos=None)`: agrega el paso a la lista recibida, o a una lista nueva si no se pasó ninguna. Cada llamada sin lista debe empezar de cero.
- `copia_independiente(config)`: devuelve una copia que se pueda modificar, incluso en sus listas internas, sin tocar el original.
- `crear_contador()`: devuelve una función que, cada vez que se llama, devuelve 1, 2, 3... Cada contador creado es independiente. Usa una clausura con `nonlocal`.

Plantilla para empezar

```

1 def agregar_paso(paso, pasos=[]):
2     pasos.append(paso)
3     return pasos
4
5
6 def copia_independiente(config):
7     return dict(config)
8
9
10 def crear_contador():
11     """c = crear_contador(); c() -> 1; c() -> 2"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué compara el operador is?

- a) Valor
- b) Identidad (mismo objeto en memoria)
- c) Hash
- d) Tipo

5 Profiling y optimización

Primero mide, después optimiza. `timeit` mide fragmentos pequeños; `cProfile` muestra dónde se va el tiempo en un programa completo; `tracemalloc` rastrea memoria.

La mayor ganancia casi siempre viene del algoritmo y la estructura de datos: pasar de $O(n)$ a $O(1)$ supera cualquier micro-optimización.

Después: evitar trabajo repetido (caché), usar built-ins implementados en C y vectorizar con NumPy.

El concepto a fondo

Primero mide, después optimiza. La intuición sobre qué es lento suele fallar. `timeit` mide fragmentos pequeños repitiéndolos muchas veces; `cProfile` muestra en qué funciones se va el tiempo de un programa completo.

La mayor ganancia casi siempre viene de elegir la estructura de datos correcta. Buscar en una lista (`x in lista`) revisa elemento por elemento: $O(n)$. Buscar en un set o diccionario usa una tabla hash: $O(1)$ en promedio, sin importar el tamaño.

Dos bucles anidados sobre los mismos datos son $O(n^2)$: con 10.000 elementos, 100 millones de comparaciones. Reemplazar el bucle interno por un set lo vuelve $O(n)$: 10.000 operaciones.

Otras técnicas: evitar trabajo repetido (calcular una vez fuera del bucle), usar funciones integradas implementadas en C (`sum`, `sorted`, `min`), memorizar con `lru_cache` y generadores para no crear listas gigantes. En esta práctica el evaluador cuenta cuántas comparaciones hace tu código.

Ejemplo

```

1 import timeit
2
3 lista = list(range(10_000))
4 conjunto = set(lista)
5 t_lista = timeit.timeit(lambda: 9_999 in lista, number=1_000)
6 t_set = timeit.timeit(lambda: 9_999 in conjunto, number=1_000)
7 print(f"lista: {t_lista:.4f}s | set: {t_set:.6f}s")
8 print(f"el set es ~{t_lista / t_set:.0f}x más rápido")

```

Salida real (los tiempos varían en cada equipo)

```

lista: 0.0603s | set: 0.000063s
el set es ~953x más rápido

```

El ejemplo, línea por línea

1. Se crea una lista y un set con los mismos 10.000 números.
2. `timeit` repite mil veces la búsqueda del último elemento en cada uno.
3. El set es cientos de veces más rápido porque no recorre los elementos.

Consejos

- `python -m cProfile -s cumtime script.py` ordena por tiempo acumulado.
- La optimización prematura es la raíz de muchos males: primero que funcione, luego que sea rápido.

Ejercicio

Perfila uno de tus scripts con `cProfile`, identifica las 3 funciones más lentas y mejora al menos una.

Errores frecuentes

Error frecuente	Cómo resolverlo
Optimizar sin medir	Puedes pasar horas acelerando una parte que solo consume el 1 % del tiempo. Perfila primero.
Convertir a set dentro del bucle	<code>if x in set(lista)</code> dentro de un bucle crea el set en cada vuelta: créalo una vez, antes.

Práctica evaluable

De $O(n^2)$ a $O(n)$ · Pieza del proyecto: Utilidades optimizadas

Esta versión funciona, pero compara cada elemento con todos los demás ($O(n^2)$). Reescríbela en $O(n)$ usando sets:

- `duplicados(items)`: lista de los elementos que aparecen más de una vez, cada uno una sola vez, en el orden en que se repiten por primera vez.
- `faltantes(requeridos, disponibles)`: lista de los requeridos que no están en disponibles, en el orden de requeridos.

El evaluador usa objetos que cuentan cada comparación `==`: la versión lenta hace miles; la tuya debe hacer muy pocas.

Plantilla para empezar

```

1 def duplicados(items):
2     resultado = []
3     for i, x in enumerate(items):
4         for y in items[:i]:
5             if x == y and x not in resultado:
6                 resultado.append(x)
7                 break
8     return resultado
9
10
11 def faltantes(requeridos, disponibles):
12     return [x for x in requeridos if x not in disponibles]

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Complejidad promedio de `x in set`?

- a) $O(n)$
- b) $O(\log n)$
- c) $O(1)$
- d) $O(n^2)$

6 Concurrency avanzada

`asyncio.TaskGroup` (3.11+) es la forma estructurada de lanzar tareas: si una falla, cancela las demás y propaga un `ExceptionGroup`.

Los semáforos limitan cuántas tareas acceden a un recurso a la vez (por ejemplo, máximo 3 conexiones simultáneas a una API).

Para mezclar código bloqueante con async usa `asyncio.to_thread`; para CPU, un `ProcessPoolExecutor`.

El concepto a fondo

`asyncio.TaskGroup` (Python 3.11+) es la forma estructurada de lanzar tareas: todas terminan dentro del bloque `async with` y, si una falla, se cancelan las demás y el error se propaga. Junto con `Semaphore` permite limitar la concurrencia de forma segura.

Para cálculos que usan la CPU, los hilos no sirven en CPython por el GIL (un solo hilo ejecuta bytecode a la vez). `concurrent.futures.ProcessPoolExecutor` reparte el trabajo entre procesos, cada uno con su intérprete y su núcleo.

El patrón detrás es map-reduce: dividir los datos en lotes (map: cada trabajador procesa el suyo) y combinar los resultados parciales (reduce). Dividir bien importa: lotes parejos evitan que un trabajador termine mucho después que el resto.

En esta práctica escribes las dos mitades del patrón, sin procesos reales: `dividir_en_lotes` y `combinar`. En tu IDE puedes conectarlas con `ProcessPoolExecutor().map(funcion, lotes)` y ver la aceleración.

Ejemplo

```

1  import asyncio
2
3  async def tarea(i, sem):
4      async with sem:
5          await asyncio.sleep(0.05)
6          return i * i
7
8  async def main():
9      sem = asyncio.Semaphore(3)
10     async with asyncio.TaskGroup() as tg:
11         tareas = [tg.create_task(tarea(i, sem)) for i in range(6)]
12     print([t.result() for t in tareas])
13
14  asyncio.run(main())

```

Salida real

```
[0, 1, 4, 9, 16, 25]
```

El ejemplo, línea por línea

1. El Semaphore(3) permite como máximo tres tareas a la vez.
2. TaskGroup crea las seis tareas y espera a que terminen todas.
3. Cada t.result() da el valor que devolvió su tarea.

Consejos

- Captura grupos de errores con except* TipoDeError.
- Usa asyncio.timeout(seg) para limitar cuánto puede tardar un bloque.

Ejercicio

Crea un crawler que visite 50 URLs simuladas con máximo 5 conexiones simultáneas y manejo de fallos parciales.

Errores frecuentes

Error frecuente	Cómo resolverlo
Usar hilos para cálculos pesados	Por el GIL no se acelera: usa procesos (ProcessPoolExecutor).
Lotes muy desparejos	Si un lote tiene el doble que otro, el total lo marca el más lento. Reparte el resto de a uno.

Práctica evaluable**Map-reduce: dividir y combinar** · Pieza del proyecto: **Reparto de trabajo**

- `dividir_en_lotes(items, trabajadores)`: reparte la lista en `trabajadores` lotes consecutivos, sin perder el orden, cuyos tamaños difieran como mucho en 1 (los primeros llevan el elemento extra). Nunca devuelve lotes vacíos: si hay menos elementos que `trabajadores`, usa tantos lotes como elementos. Con una lista vacía, devuelve `[]`.
- `combinar(resultados)`: recibe una lista de diccionarios de conteos parciales y devuelve un diccionario con la suma por clave.

Plantilla para empezar

```

1 def dividir_en_lotes(items, trabajadores):
2     """dividir_en_lotes(list(range(10)), 3) -> [[0,1,2,3], [4,5,6], [7,8,9]]"""
3
4
5 def combinar(resultados):
6     """combinar([{"a": 1}, {"a": 2, "b": 1}]) -> {"a": 3, "b": 1}"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si una tarea de un TaskGroup falla...

- Se reintenta sola
- Las demás siguen
- Se ignora
- Se cancelan las demás y se propaga un `ExceptionGroup`

7**Patrones de diseño pythónicos**

Muchos patrones clásicos se simplifican en Python: Strategy es pasar una función, Singleton es un módulo, Iterator es un generador.

`typing.Protocol` permite tipado estructural: cualquier clase con los métodos correctos cumple el contrato, sin heredar. Es duck typing verificable por `mypy`.

La inyección de dependencias (pasar colaboradores como parámetros) desacopla el código y lo vuelve fácil de probar.

El concepto a fondo

Muchos patrones clásicos se simplifican en Python porque las funciones son objetos: Strategy es pasar una función como argumento, Command es guardar una función para llamarla después, Singleton suele ser simplemente un módulo, e Iterator es un generador.

Observer (o publicador/suscriptor) desacopla quien produce un evento de quien reacciona: el emisor no sabe quién escucha. Un bus de eventos guarda, para cada nombre de evento, la lista de funciones suscritas y las llama al emitir.

`typing.Protocol` define una interfaz por su forma (duck typing estático): cualquier clase con los métodos indicados la cumple, sin heredar de nada. `Mypy` verifica que los objetos que pasas la respeten.

Inyección de dependencias: en lugar de que un componente cree lo que necesita (un notificador, una base de datos), lo recibe por parámetro. Así puedes cambiarlo por otro, o por uno falso en los tests, sin tocar el componente.

Ejemplo

```

1  from typing import Protocol
2
3  class Notificador(Protocol):
4      def enviar(self, msg: str) -> str: ...
5
6  class Email:
7      def enviar(self, msg: str) -> str:
8          return f"[email] {msg}"
9
10 class Telegram:
11     def enviar(self, msg: str) -> str:
12         return f"[telegram] {msg}"
13
14 def alertar(canal: Notificador, msg: str) -> None:
15     print(canal.enviar(msg))
16
17 for canal in (Email(), Telegram()):
18     alertar(canal, "Stock bajo")

```

Salida real

```

[email] Stock bajo
[telegram] Stock bajo

```

El ejemplo, línea por línea

1. Notificador es un Protocol: describe que hace falta un método enviar.
2. Email y Telegram lo cumplen sin heredar.
3. alertar recibe cualquier canal: es Strategy e inyección de dependencias a la vez.

Consejos

- Depende de abstracciones (protocolos), no de implementaciones concretas.
- No apliques un patrón porque existe: aplícalo cuando el problema lo pida.

Ejercicio

Refactoriza tu monitor de precios para que las fuentes de datos y los notificadores sean intercambiables vía Protocol.

Errores frecuentes

Error frecuente	Cómo resolverlo
Todos los buses comparten suscriptores	Creaste el diccionario como atributo de clase. Créalo en <code>__init__</code> (<code>self._suscriptores = {}</code>).
Un decorador de registro que borra la función	El decorador debe devolver la función original para que siga usable por su nombre.

Práctica evaluable**Un bus de eventos (Observer) · Pieza del proyecto: Bus de eventos**

Crea la clase EventBus:

- `suscribir(evento, funcion)`: registra la función para ese evento y la devuelve.
- `al(evento)`: un decorador que suscribe la función decorada y la deja intacta: `@bus.al("tarea_lista")`.
- `desuscribir(evento, funcion)`: la quita; si no estaba, no hace nada.
- `emitir(evento, *args, **kwargs)`: llama a todas las funciones del evento, en orden de suscripción, y devuelve la lista de sus resultados. Sin suscriptores, devuelve `[]`.

Cada bus es independiente de los demás.

Plantilla para empezar

```
1 class EventBus:
2     """Publicador/suscriptor: bus.emitir("evento", datos) avisa a quienes se
   ↪ suscribieron."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

`typing.Protocol` permite...

- Crear metaclasses
- Herencia múltiple
- Ejecutar más rápido
- Tipado estructural sin herencia

8 Testing avanzado

Las fixtures de `pytest` preparan datos y recursos reutilizables; `@pytest.mark.parametrize` ejecuta un test con muchos casos.

`unittest.mock` reemplaza dependencias externas (APIs, bases de datos, reloj) para probar tu lógica de forma aislada, rápida y determinista.

Las pruebas basadas en propiedades (hypothesis) generan cientos de entradas para encontrar casos borde que no se te habían ocurrido. `coverage` mide qué código ejecutan tus tests.

El concepto a fondo

Las fixtures de `pytest` preparan datos y recursos reutilizables: una función decorada con `@pytest.fixture` se inyecta en los tests que la nombran como parámetro. `@pytest.mark.parametrize` ejecuta el mismo test con muchos valores.

Un doble de prueba reemplaza una dependencia real (una API, el correo, el reloj) por algo controlado. Un stub devuelve respuestas fijas; un fake es una implementación simple que funciona (una lista en lugar de un servicio de mensajes); un mock además registra cómo fue usado para verificarlo.

`unittest.mock.Mock` crea objetos que aceptan cualquier llamada y la registran: `assert_called_once_with(...)` verifica que se usó como esperabas. Úsalo con criterio: probar el comportamiento visible es más robusto que verificar cada llamada interna.

Como en el Nivel 2, el evaluador correrá tus tests contra la función correcta y contra versiones con errores. Un buen conjunto cubre el camino feliz, el camino de error y los reintentos.

Ejemplo

```

1  from unittest.mock import Mock
2
3  def precio_final(api, codigo):
4      base = api.obtener_precio(codigo)
5      return round(base * 1.16, 2)
6
7  api_falsa = Mock()
8  api_falsa.obtener_precio.return_value = 100
9  print(precio_final(api_falsa, "TUB0-1"))
10 api_falsa.obtener_precio.assert_called_once_with("TUB0-1")
11 print("Mock verificado")

```

Salida real

```

116.0
Mock verificado

```

El ejemplo, línea por línea

1. `Mock()` crea una API falsa.
2. `return_value = 100` fija lo que devuelve `obtener_precio`.
3. Se verifica el cálculo y que la API se llamó una vez con el código correcto.

Consejos

- Un buen test es rápido, aislado, repetible y falla por una sola razón.
- Mockea en el borde del sistema (red, disco, tiempo), no tu propia lógica.

Ejercicio

Alcanza 90 % de cobertura en tu monitor de precios usando fixtures, parametrize y mocks.

Errores frecuentes

Error frecuente	Cómo resolverlo
Tests que dependen de internet o de la hora	Fallan al azar. Reemplaza esas dependencias por dobles.
Mockear todo	Si mockeas la propia función que pruebas, el test no verifica nada. Reemplaza solo las dependencias externas.

Práctica evaluable**Tests con dobles de prueba · Pieza del proyecto: Tests del ejecutor de tareas**

La función `ejecutar_tarea(tarea, notificar, reintentos=1)` ya existe:

- llama a `tarea()`; si funciona, llama a `notificar("ok: RESULTADO")` y devuelve `True`;
- si `tarea()` lanza un error, la reintentará hasta `reintentos` veces más; si sigue fallando, llama a `notificar("error: MENSAJE")` y devuelve `False`.

Escribe al menos 3 tests (`test_...`) usando un notificador falso (por ejemplo, el método `append` de una lista) y tareas falsas que cubran: el caso exitoso, el error definitivo y una tarea que falla una vez y luego funciona.

Tus tests deben pasar con la función correcta y detectar cuatro versiones con errores.

Datos que prepara el evaluador (también prueba con otros valores)

```

1 def ejecutar_tarea(tarea, notificar, reintentos=1):
2     """Ejecuta tarea(); reintentará ante errores y avisa el resultado con
   ↪ notificar(mensaje)."""
3     for intento in range(reintentos + 1):
4         try:
5             resultado = tarea()
6         except Exception as e:
7             if intento == reintentos:
8                 notificar(f"error: {e}")
9                 return False
10        else:
11            notificar(f"ok: {resultado}")
12            return True

```

Plantilla para empezar

```

1 def test_tarea_exitosa_avisa_ok():
2     mensajes = []
3     resultado = ejecutar_tarea(lambda: 42, mensajes.append)
4     assert resultado is True
5     assert mensajes == ["ok: 42"]
6
7
8 # Escribe el test del error definitivo y el del reintentado exitoso

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Para qué sirve un mock?

- Medir cobertura
- Acelerar Python
- Generar documentación
- Reemplazar dependencias para aislar lo que se prueba

9 Rendimiento nativo

Cuando Python puro no alcanza: NumPy vectoriza operaciones sobre arrays en código C; Numba compila funciones numéricas con JIT; Cython y las extensiones en C o Rust (PyO3) llevan partes críticas a código nativo.

Vectorizar significa operar sobre arrays completos en lugar de recorrerlos con bucles Python.

Regla práctica: perfila, aísla el 5 % del código que consume el 95 % del tiempo y optimiza solo eso.

El concepto a fondo

Cuando Python puro no alcanza, se delega el trabajo pesado a código compilado. NumPy guarda números en bloques contiguos de memoria y aplica operaciones a todo el arreglo de una vez (vectorización), cientos de veces más rápido que un bucle de Python.

Numba compila funciones numéricas a código máquina con un decorador, Cython agrega tipos para compilar módulos, y librerías como Polars están escritas en Rust. La idea común: el bucle no ocurre en Python sino en código nativo.

Aun sin librerías externas, las funciones integradas están implementadas en C: `sum`, `min`, `max`, `sorted`, `zip`, `map` y `any` recorren datos mucho más rápido que un `for` con índices escrito a mano. Pensar en "operaciones sobre colecciones completas" es el primer paso hacia la vectorización.

`zip(*matriz)` transpone filas en columnas, `sum(x * y for x, y in zip(a, b))` calcula un producto punto y `min/max` normalizan valores. En tu IDE, compara estas versiones con las de NumPy (`np.dot`, `.T`) para ver la diferencia de velocidad.

Ejemplo

```
1 import numpy as np
2
3 precios = np.array([10.0, 20.0, 30.0])
4 cantidades = np.array([3, 1, 2])
5 subtotales = precios * cantidades
6 print(subtotales, subtotales.sum())
7 print(np.where(precios > 15, "caro", "ok"))
```

Salida real

```
[30. 20. 60.] 110.0
['ok' 'caro' 'caro']
```

El ejemplo, línea por línea

1. Se crean dos arreglos de NumPy.
2. `precios * cantidades` multiplica elemento a elemento sin bucles.
3. `np.where` elige un valor u otro según la condición, para todo el arreglo.

Consejos

- Un bucle Python sobre un millón de elementos es decenas de veces más lento que su versión NumPy.
- Polars es una alternativa a pandas escrita en Rust, muy rápida para datos grandes.

Ejercicio

Calcula estadísticas de un millón de números con bucles puros y con NumPy, y compara tiempos con `timeit`.

Errores frecuentes

Error frecuente	Cómo resolverlo
Bucles de Python sobre arreglos de NumPy	Pierdes la ventaja: usa operaciones vectorizadas sobre el arreglo completo.
zip corta en silencio la lista más larga	Si las longitudes pueden diferir, verifícalas antes (o usa <code>zip(..., strict=True)</code> en Python 3.10+).

Práctica evaluable

Pensar en colecciones completas · Pieza del proyecto: **Cálculo vectorial**

Sin índices ni bucles con `range(len(...))`, usando funciones integradas:

- `producto_punto(a, b)`: suma de los productos elemento a elemento. Si las longitudes difieren, `ValueError`.
- `transponer(matriz)`: filas $\leftrightarrow$ columnas, como lista de listas.
- `normalizar(valores)`: escala cada valor al rango 0-1 con $(v - \text{mínimo}) / (\text{máximo} - \text{mínimo})$. Si todos son iguales, devuelve una lista de 0.0; con una lista vacía, `[]`.

Plantilla para empezar

```

1 def producto_punto(a, b):
2     """producto_punto([1, 2, 3], [4, 5, 6]) -> 32"""
3
4
5 def transponer(matriz):
6     """[[1, 2, 3], [4, 5, 6]] -> [[1, 4], [2, 5], [3, 6]]"""
7
8
9 def normalizar(valores):
10    """[10, 20, 30] -> [0.0, 0.5, 1.0]"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Vectorizar significa...

- Operar sobre arrays completos en código compilado, sin bucles Python
- Usar hilos
- Usar listas
- Dibujar vectores

10 Arquitectura y calidad

Arquitectura limpia: el núcleo (reglas de negocio) no depende de frameworks, bases de datos ni interfaces; esos detalles se conectan en los bordes.

Calidad automatizada: ruff (estilo y errores), mypy (tipos), pytest + coverage (pruebas) y pre-commit para correrlos antes de cada commit.

La integración continua (CI) ejecuta todo en cada push. Si el pipeline está en verde, el código está listo para integrarse.

El concepto a fondo

Arquitectura limpia: el núcleo (las reglas de negocio) no depende de frameworks, bases de datos ni interfaces; son ellos los que dependen del núcleo. Así puedes cambiar la base de datos o la interfaz sin reescribir la lógica, y probarla sin ninguna de las dos.

La raíz de composición es el único lugar donde se arma todo: lee la configuración, crea los objetos concretos y los conecta. El resto del código recibe sus dependencias ya construidas.

Valida la configuración al arrancar y falla rápido con mensajes claros: es mucho mejor un error inmediato que diga "Plugins desconocidos: inverttr" que un fallo misterioso horas después.

La calidad se automatiza: ruff (estilo y errores comunes), mypy (tipos) y pytest (comportamiento) corriendo en integración continua (CI) en cada cambio. El archivo de ejemplo de la lección configura exactamente eso en GitHub Actions.

Ejemplo (yaml)

```

1  # .github/workflows/ci.yml
2  name: CI
3  on: [push, pull_request]
4  jobs:
5    test:
6      runs-on: ubuntu-latest
7      steps:
8        - uses: actions/checkout@v4
9        - uses: actions/setup-python@v5
10         with:
11           python-version: "3.12"
12         - run: pip install -e ".[dev]"
13         - run: ruff check . && mypy src && pytest -q

```

Resultado esperado

```
✓ test (ruff: 0 errores · mypy: Success · pytest: 42 passed)
```

El ejemplo, línea por línea

1. El workflow corre en cada push y pull request.
2. Instala Python 3.12 y el proyecto con sus dependencias de desarrollo.
3. La última línea ejecuta ruff, mypy y pytest: si algo falla, el cambio queda marcado.

Consejos

- Documenta las decisiones de arquitectura en ADRs (Architecture Decision Records).
- El mejor código es el que tu equipo entiende dentro de seis meses.

Ejercicio

Configura ruff, mypy, pre-commit y un workflow de CI para tu proyecto del nivel 3.

Errores frecuentes

Error frecuente	Cómo resolverlo
Configuración inválida descubierta tarde	Valida todo al crear la aplicación y lanza errores con mensajes concretos.
Lógica de negocio mezclada con la entrada/salida	Si para probar una regla necesitas un archivo o una red, sepáralas.

Práctica evaluable**La raíz de composición** · Pieza del proyecto: **Aplicación y raíz de composición**

Ya existe REGISTRO, un diccionario {nombre: función}.

- `App(pasos, reintentos=0)`: guarda los pasos y los reintentos. `procesar(dato)` aplica los pasos en orden y `procesar_lote(datos)` devuelve la lista de resultados.
- `crear_app(config, registro)`: arma la App a partir de un diccionario de configuración como `{"plugins": ["recortar", "mayus"], "reintentos": 2}`. Valida y falla rápido:
 - sin "plugins" → `ValueError("Falta la lista de plugins")`;
 - con nombres inexistentes → `ValueError("Plugins desconocidos: invertr, mayu")`, con todos los desconocidos en su orden;
 - reintentos (0 por defecto) debe ser un entero entre 0 y 10; si no, `ValueError("reintentos debe ser un entero entre 0 y 10")`.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 REGISTRO = {"mayus": str.upper, "recortar": str.strip, "invertir": lambda s:
  ↳ s[::-1]}
```

Plantilla para empezar

```
1 class App:
2     """Núcleo: aplica pasos a los datos. No sabe de dónde vienen."""
3
4
5     def crear_app(config, registro):
6         """Raíz de composición: valida la configuración y arma la App."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué herramienta verifica tipos de forma estática?

- a) pip
- b) pytest
- c) mypy
- d) ruff

Proyecto integrador

Framework de automatización con plugins

Una librería publicable donde cada tarea (descargar, transformar, exportar, notificar) es un plugin que se registra solo, con pipeline asíncrono, tipado estricto y CI.

Paso a paso

1. Diseña el núcleo con arquitectura limpia y contratos vía Protocol.
2. Implementa el registro de plugins con `__init_subclass__` y validación con descriptores.
3. Ejecuta pipelines con TaskGroup, semáforos y manejo de ExceptionGroup.
4. Perfila y optimiza el cuello de botella (NumPy o caché).
5. Cobertura > 90 % con fixtures, parametrize, mocks e hypothesis.
6. Publica en TestPyPI con CI (ruff, mypy, pytest) y documentación con MkDocs.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Puedo explicar y usar: el modelo de datos a fondo
- ☐ Puedo explicar y usar: descriptores y properties
- ☐ Puedo explicar y usar: metaprogramación
- ☐ Puedo explicar y usar: internals de cpython
- ☐ Puedo explicar y usar: profiling y optimización
- ☐ Puedo explicar y usar: concurrencia avanzada
- ☐ Puedo explicar y usar: patrones de diseño pythónicos
- ☐ Puedo explicar y usar: testing avanzado
- ☐ Puedo explicar y usar: rendimiento nativo
- ☐ Puedo explicar y usar: arquitectura y calidad
- ☐ Aprobé las 10 prácticas evaluables y mi proyecto ejecuta la demo
- ☐ Terminé el proyecto "Framework de automatización con plugins"

Tu proyecto del nivel, pieza a pieza

Framework de automatización con plugins

Un pequeño framework para encadenar pasos de procesamiento, configurado con validación automática, extensible con plugins que se registran solos y con eventos para avisar resultados. Cada lección del Nivel 4 aporta una pieza del motor.

Cada práctica evaluable del nivel es una pieza de este programa. En la app, la sección **Mi proyecto** (desde el Árbol o el Inicio) une tus soluciones aprobadas en un solo archivo, `mini_framework.py`, te deja ejecutar la demo en el navegador cuando están todas y descargarlo en .zip (con un README) para abrirlo en tu IDE. Las piezas pendientes se reemplazan por versiones provisionales, así que puedes descargarlo en cualquier momento.

Las piezas

Lección	Pieza	Práctica
4.1 El modelo de datos a fondo	Pipeline (modelo de datos)	Un Pipeline que se comporta como un tipo nativo
4.2 Descriptores y properties	Config con descriptores	Configuración validada con descriptores
4.3 Metaprogramación	Registro automático de plugins	Plugins que se registran solos
4.4 Internals de CPython	Utilidades del motor	Referencias, valores por defecto y clausuras
4.5 Profiling y optimización	Utilidades optimizadas	De $O(n^2)$ a $O(n)$
4.6 Concurrencia avanzada	Reparto de trabajo	Map-reduce: dividir y combinar
4.7 Patrones de diseño pythónicos	Bus de eventos	Un bus de eventos (Observer)
4.8 Testing avanzado	Tests del ejecutor de tareas	Tests con dobles de prueba
4.9 Rendimiento nativo	Cálculo vectorial	Pensar en colecciones completas
4.10 Arquitectura y calidad	Aplicación y raíz de composición	La raíz de composición

El programa principal

Este es el código que Pythonia agrega al final del archivo: no tienes que escribirlo, pero vale la pena leerlo, porque muestra cómo se conectan tus piezas. `demo()` hace un recorrido automático y el bloque `if __name__ == "__main__":` es lo que corre al ejecutar el archivo en tu IDE.

```

1  # ===== Ensamble final (escrito por Pythonia, usa tus piezas)
   ↳ =====
2  def ejecutar_tarea(tarea, notificar, reintentos=1):
3      """Ejecuta tarea(); reintentos ante errores y avisa el resultado con
   ↳ notificar(mensaje)."""
4      for intento in range(reintentos + 1):
5          try:
6              resultado = tarea()
7          except Exception as e:
8              if intento == reintentos:
9                  notificar(f"error: {e}")
10                 return False
11             else:
12                 notificar(f"ok: {resultado}")
13                 return True
14
15
16 class Invertir(Plugin, nombre="invertir"):
17     def ejecutar(self, dato):
18         return dato[::-1]
19
20
21 def demo():
22     """Recorrido automático por todas tus piezas."""
23     print("Plugins registrados:", sorted(Plugin.registro))
24     registro = {n: Plugin.crear(n).ejecutar for n in Plugin.registro}
25     config = Config(reintentos=2, timeout=30)
26     app = crear_app({"plugins": ["recortar", "mayus", "invertir"], "reintentos":
   ↳ config.reintentos}, registro)
27     bus = EventBus()
28
29     @bus.al("procesado")
30     def avisar(dato):
31         return f"listo: {dato}"
32
33     for texto in [" hola mundo ", " python "]:
34         print(bus.emitir("procesado", app.procesar(texto)))
35     pipe = Pipeline(str.strip, str.title) + Pipeline(lambda s: s + "!")
36     print(len(pipe), "pasos ->", repr(pipe(" pythonia maestro ")))
37     print("Lotes para 3 trabajadores:", dividir_en_lotes(list(range(10)), 3))

```

```

38     print("Conteo combinado:", combinar([{"cable": 1}, {"cable": 2, "rodillo": 1}]))
39     print("Duplicados:", duplicados(["a", "b", "a", "c", "b"]), "| faltan:",
↵ faltantes(["a", "x", "b"], ["a", "b"]))
40     print("Producto punto:", producto_punto([1, 2, 3], [4, 5, 6]), "|
↵ normalizado:", normalizar([10, 20, 30]))
41     contar = crear_contador()
42     contar()
43     print("Ejecuciones contadas:", contar(), "| pasos independientes:",
↵ agregar_paso("a"), agregar_paso("b"))
44     try:
45         Config(reintentos=50)
46     except ValueError as e:
47         print("Configuración rechazada:", e)
48     print("— Tus tests —")
49     for nombre, prueba in list(globals().items()):
50         if nombre.startswith("test_") and callable(prueba):
51             prueba()
52             print("✓", nombre)
53
54
55 if __name__ == "__main__":
56     demo()

```

Ejercicios con lente de especialización

Cada concepto del nivel aplicado a tu ruta. Elige la columna de tu ruta principal; si sigues el Modo Pythonista Total, resuelve las tres.

Automatización de oficina

Lección	Ejercicio con lente
1. El modelo de datos a fondo	Haz que tu clase Planilla soporte <code>len()</code> , iteración por filas e indexación <code>planilla['A1']</code> .
2. Descriptores y properties	Crea un descriptor <code>ColumnaNumerica</code> que convierta y valide los valores al asignarlos.
3. Metaprogramación	Registra automáticamente cada tipo de tarea del robot (descargar, limpiar, enviar) con <code>__init_subclass__</code> .
4. Internals de CPython	Mide con <code>tracemalloc</code> la memoria de cargar una planilla completa contra recorrerla con un generador.
5. Profiling y optimización	Perfila el robot con <code>cProfile</code> y optimiza el paso más lento.
6. Concurrencia avanzada	Procesa 50 planillas con <code>TaskGroup</code> y un semáforo de 5, tolerando fallos parciales.
7. Patrones de diseño pythónicos	Haz intercambiables los destinos del reporte (correo, carpeta, Telegram) con un Protocol.
8. Testing avanzado	Mockea <code>smtplib</code> y el sistema de archivos para probar el robot sin enviar correos reales.
9. Rendimiento nativo	Reemplaza el bucle de totales por <code>pandas</code> o <code>DuckDB</code> y compara tiempos con 100.000 filas.
10. Arquitectura y calidad	Configura CI para tu robot: <code>ruff</code> , <code>mypy</code> y <code>pytest</code> en cada push.

Videojuegos

Lección	Ejercicio con lente
1. El modelo de datos a fondo	Implementa <code>__add__</code> y <code>__mul__</code> en tu propia clase <code>Vector</code> y úsala en la física del juego.
2. Descriptores y properties	Crea un descriptor <code>Limitado(min, max)</code> para que vida y energía nunca salgan de su rango.
3. Metaprogramación	Registra cada tipo de power-up con <code>__init_subclass__</code> para generarlos al azar.
4. Internals de CPython	Compara con <code>dis</code> el bytecode de tu función de colisión antes y después de optimizarla.
5. Profiling y optimización	Perfila un cuadro del juego y baja el tiempo de actualización a menos de 16 ms.
6. Concurrencia avanzada	Calcula la ruta de los enemigos (pathfinding) en un proceso aparte para no congelar el juego.
7. Patrones de diseño pythónicos	Implementa el patrón <code>Command</code> para las entradas y permite reasignar teclas.
8. Testing avanzado	Usa <code>hypothesis</code> para comprobar que ninguna secuencia de movimientos saca al jugador del mapa.

Lección	Ejercicio con lente
9. Rendimiento nativo	Simula 10.000 partículas con NumPy en lugar de una lista de objetos y compara los FPS.
10. Arquitectura y calidad	Organiza el juego en capas (motor, lógica, presentación) y configura CI con tests de la lógica.

Bases de datos

Lección	Ejercicio con lente
1. El modelo de datos a fondo	Haz que un objeto Tabla soporte len(), in e iteración usando consultas por debajo.
2. Descriptores y properties	Crea un descriptor Columna que valide tipos al asignar, como hace un ORM.
3. Metaprogramación	Construye un mini-ORM: modelos que generen su CREATE TABLE a partir de sus anotaciones.
4. Internals de CPython	Compara con tracemalloc la memoria de fetchall() contra iterar el cursor fila a fila.
5. Profiling y optimización	Usa EXPLAIN QUERY PLAN y timeit para acelerar la consulta más lenta de tu proyecto.
6. Concurrencia avanzada	Simula 20 escrituras concurrentes, observa los bloqueos y resuélvelos con modo WAL y reintentos.
7. Patrones de diseño pythónicos	Aplica los patrones Repository y Unit of Work para separar negocio y persistencia.
8. Testing avanzado	Testea el repositorio con fixtures parametrizadas contra SQLite y la versión en memoria.
9. Rendimiento nativo	Compara el mismo análisis en pandas, SQLite y DuckDB sobre un millón de filas.
10. Arquitectura y calidad	Agrega migraciones con Alembic y un pipeline de CI que las pruebe.

Anexo: las 13 rutas de especialización

Ruta	Desde	Módulos y proyecto
Ciencia de datos NumPy, pandas, Polars, Matplotlib, Seaborn, Jupyter	Nivel 2	NumPy y arrays, pandas: limpieza y transformación, Agrupaciones, pivotes y fechas, Visualización, Estadística aplicada, Reportes automáticos. <i>Proyecto: Análisis exploratorio completo de un dataset de ventas con informe final.</i>
Machine learning e IA scikit-learn, PyTorch, Hugging Face, APIs de LLM	Nivel 3	Fundamentos y preparación de datos, Regresión y clasificación, Evaluación y validación, Redes neuronales con PyTorch, NLP y transformers, Apps con LLM y RAG. <i>Proyecto: Modelo de predicción de demanda servido como API.</i>
Desarrollo web Flask, Django, FastAPI, SQLAlchemy, Jinja2	Nivel 2	HTTP y la web, Flask: rutas y plantillas, Bases de datos con ORM, APIs REST con FastAPI, Autenticación y seguridad, Despliegue. <i>Proyecto: API REST con autenticación y panel web de administración.</i>
Automatización de oficina openpyxl, pathlib, python-docx, ReportLab, schedule, Playwright	Nivel 1	Archivos y carpetas, Excel con openpyxl, Word y PDF, Correo automático, Automatización web, Tareas programadas. <i>Proyecto: Robot que consolida planillas, genera un PDF y lo envía por correo cada semana.</i>
Bots y mensajería python-telegram-bot, discord.py, httpx	Nivel 2	Cómo funciona un bot, Comandos y respuestas, Teclados y botones, Persistencia de datos, Webhooks, Despliegue 24/7. <i>Proyecto: Bot de Telegram que consulta datos de negocio y envía alertas.</i>
Dashboards y BI Streamlit, Plotly, Dash	Nivel 2	Gráficos interactivos con Plotly, Primera app en Streamlit, Filtros y estado, Caché y rendimiento, Multi-página y autenticación, Publicación. <i>Proyecto: Dashboard de KPIs con filtros por fecha, sucursal y producto.</i>
Bases de datos sqlite3, SQLAlchemy, psycopy, DuckDB	Nivel 2	SQL esencial, sqlite3 desde Python, Modelado de datos, SQLAlchemy ORM, PostgreSQL, Análisis con DuckDB. <i>Proyecto: Sistema de inventario con base de datos relacional y reportes.</i>
DevOps y scripting subprocess, Typer, Docker SDK, Fabric, boto3	Nivel 2	Scripts de sistema, CLIs con Typer, Docker y contenedores, Automatizar servidores, Nube (AWS/GCP), Monitoreo. <i>Proyecto: CLI que despliega una app en contenedor y verifica su salud.</i>
Videojuegos pygame, arcade	Nivel 1	Bucle de juego, Sprites y animación, Colisiones, Sonido, Niveles y estados, Publicar tu juego. <i>Proyecto: Juego de plataformas 2D con tres niveles.</i>
Apps de escritorio tkinter, PySide6, PyInstaller	Nivel 2	Ventanas y widgets, Layouts, Eventos, Formularios y validación, Apps con datos, Empaquetar un .exe. <i>Proyecto: App de escritorio para registrar y consultar pedidos.</i>
Ciberseguridad defensiva hashlib, secrets, cryptography, scapy	Nivel 3	Fundamentos y ética, Hashing y contraseñas, Cifrado, Redes y sockets, Análisis de logs, Auditoría de tu propia red. <i>Proyecto: Analizador de logs que detecta accesos sospechosos en tu propia infraestructura.</i>

Ruta	Desde	Módulos y proyecto
IoT y hardware MicroPython, gpiozero, paho-mqtt	Nivel 2	MicroPython en placas, Sensores y actuadores, Raspberry Pi, MQTT, Datos en tiempo real, Proyecto conectado. <i>Proyecto: Estación de monitoreo de temperatura con alertas.</i>
Computación científica SciPy, SymPy, NumPy, Jupyter	Nivel 2	Álgebra simbólica, Cálculo numérico, Optimización, Simulaciones, Señales, Cuadernos reproducibles. <i>Proyecto: Simulación física con visualización animada.</i>

Solucionario de autoevaluación

- 1. El modelo de datos a fondo:** c) Evita el `__dict__` por instancia y ahorra memoria. Los atributos se guardan en espacios fijos en lugar de un diccionario.
- 2. Descriptores y properties:** d) Sí. `property` implementa `__get__`, `__set__` y `__delete__`.
- 3. Metaprogramación:** b) `type`. `type` crea las clases; incluso `type` es instancia de sí mismo.
- 4. Internals de CPython:** b) Identidad (mismo objeto en memoria). Dos listas iguales en valor son objetos distintos: `==` es `True`, `is` es `False`.
- 5. Profiling y optimización:** c) $O(1)$. Los sets usan tablas hash: la búsqueda es de tiempo constante en promedio.
- 6. Concurrencia avanzada:** d) Se cancelan las demás y se propaga un `ExceptionGroup`. Concurrencia estructurada: ninguna tarea queda huérfana.
- 7. Patrones de diseño pythónicos:** d) Tipado estructural sin herencia. Si tiene los métodos del protocolo, cumple el contrato.
- 8. Testing avanzado:** d) Reemplazar dependencias para aislar lo que se prueba. Así el test no depende de la red ni de servicios externos.
- 9. Rendimiento nativo:** a) Operar sobre arrays completos en código compilado, sin bucles Python. La iteración ocurre en C, no en el intérprete.
- 10. Arquitectura y calidad:** c) `mypy`. `mypy` analiza las anotaciones sin ejecutar el código.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. El modelo de datos a fondo · Un Pipeline que se comporta como un tipo nativo

Pista 1: `isinstance(indice, slice)` distingue `p[1:]` de `p[1]`.

Pista 2: `getattr(paso, "__name__", ...)` evita errores con objetos que no tienen nombre.

```

1  class Pipeline:
2      """Pipeline(paso1, paso2)(dato) == paso2(paso1(dato))"""
3
4      def __init__(self, *pasos):
5          self.pasos = list(pasos)
6
7      def __len__(self):
8          return len(self.pasos)
9
10     def __iter__(self):
11         return iter(self.pasos)
12
13     def __getitem__(self, indice):
14         if isinstance(indice, slice):
15             return Pipeline(*self.pasos[indice])
16         return self.pasos[indice]
17
18     def __add__(self, otro):
19         if not isinstance(otro, Pipeline):
20             return NotImplemented
21         return Pipeline(*self.pasos, *otro.pasos)
22
23     def __call__(self, dato):
24         for paso in self.pasos:
25             dato = paso(dato)
26         return dato
27
28     def __repr__(self):
29         nombres = " → ".join(getattr(p, "__name__", type(p).__name__) for p in
30                               ↪ self.pasos)
31         return f"Pipeline({nombres})"

```

Por qué funciona: Seis métodos especiales convierten una clase común en algo que se usa con la sintaxis de Python: `len`, `for`, índices, `+`, llamada y `repr`. Devolver `NotImplemented` deja que Python genere el `TypeError` correcto.

2. Descriptores y properties · Configuración validada con descriptores

Pista 1: Guarda el valor real con `setattr(obj, self.privado, valor)`.

Pista 2: Valida antes de guardar: así, si falla, el valor anterior queda intacto.

```

1  class Rango:
2      """Descriptor que valida que un número esté en [minimo, maximo]."""
3
4      def __init__(self, minimo, maximo):
5          self.minimo, self.maximo = minimo, maximo
6
7      def __set_name__(self, dueno, nombre):
8          self.nombre = nombre
9          self.privado = "_" + nombre
10
11     def __get__(self, obj, tipo=None):
12         if obj is None:
13             return self
14         return getattr(obj, self.privado)
15
16     def __set__(self, obj, valor):
17         if isinstance(valor, bool) or not isinstance(valor, (int, float)):
18             raise TypeError(f"{self.nombre} debe ser un número")
19         if not self.minimo <= valor <= self.maximo:
20             raise ValueError(f"{self.nombre} debe estar entre {self.minimo} y
21                               ↳ {self.maximo}")
22         setattr(obj, self.privado, valor)
23
24     class Config:
25         reintentos = Rango(0, 10)
26         timeout = Rango(1, 60)
27
28     def __init__(self, reintentos=3, timeout=10):
29         self.reintentos = reintentos
30         self.timeout = timeout

```

Por qué funciona: La regla de validación se escribe una sola vez y se reutiliza en cada atributo con una línea. Como `__init__` asigna a través del descriptor, ni siquiera el constructor puede crear una configuración inválida.

3. Metaprogramación · Plugins que se registran solos

Pista 1: Escribe `Plugin.registro`, no `cls.registro`, para que todas las subclases compartan el mismo diccionario.

Pista 2: `@classmethod` recibe la clase como primer parámetro (`cls`).

```

1 class Plugin:
2     """Base de los plugins: class MiPlugin(Plugin, nombre="mi_plugin")"""
3     registro = {}
4
5     def __init_subclass__(cls, nombre, **kw):
6         super().__init_subclass__(**kw)
7         if nombre in Plugin.registro:
8             raise ValueError(f"Ya existe un plugin llamado {nombre}")
9         cls.nombre = nombre
10        Plugin.registro[nombre] = cls
11
12    def ejecutar(self, dato):
13        raise NotImplementedError("Cada plugin debe implementar ejecutar()")
14
15    @classmethod
16    def crear(cls, nombre):
17        if nombre not in cls.registro:
18            raise KeyError(f"Plugin desconocido: {nombre}")
19        return cls.registro[nombre]()
20
21
22    class Mayusculas(Plugin, nombre="mayus"):
23        def ejecutar(self, dato):
24            return dato.upper()
25
26
27    class Recortar(Plugin, nombre="recortar"):
28        def ejecutar(self, dato):
29            return dato.strip()

```

Por qué funciona: Agregar un plugin nuevo es solo definir su clase: no hay listas que mantener a mano ni if gigantes. Así funcionan los sistemas de extensiones de muchos frameworks.

4. Internals de CPython · Referencias, valores por defecto y clausuras

Pista 1: `if pasos is None: pasos = []` crea una lista nueva en cada llamada.

Pista 2: Sin `nonlocal`, `n += 1` crearía una variable local nueva y fallaría.

```

1  import copy
2
3
4  def agregar_paso(paso, pasos=None):
5      if pasos is None:
6          pasos = []
7          pasos.append(paso)
8          return pasos
9
10
11 def copia_independiente(config):
12     return copy.deepcopy(config)
13
14
15 def crear_contador():
16     """c = crear_contador(); c() -> 1; c() -> 2"""
17     n = 0
18
19     def contar():
20         nonlocal n
21         n += 1
22         return n
23
24     return contar

```

Por qué funciona: Los tres casos son el mismo tema: a qué objeto apunta cada nombre. El valor por defecto mutable se comparte entre llamadas, `dict()` comparte las listas internas y la clausura guarda su propia `n` en una celda distinta para cada contador.

5. Profiling y optimización · De $O(n^2)$ a $O(n)$

Pista 1: Necesitas dos sets: lo ya visto y lo ya reportado como repetido.

Pista 2: Crea el set de disponibles una sola vez, antes de la comprensión.

```

1  def duplicados(items):
2      """Elementos repetidos, en orden de primera repetición.  $O(n)$ ."""
3      vistos = set()
4      repetidos = set()
5      resultado = []
6      for x in items:
7          if x in vistos and x not in repetidos:
8              repetidos.add(x)
9              resultado.append(x)
10             vistos.add(x)
11     return resultado
12
13
14 def faltantes(requeridos, disponibles):
15     """Requeridos que no están en disponibles.  $O(n)$ ."""
16     disponibles = set(disponibles)
17     return [x for x in requeridos if x not in disponibles]

```

Por qué funciona: Un set busca por hash: va directo al lugar donde estaría el elemento y solo compara con `==` si hay coincidencia de hash. Por eso la versión optimizada hace unas decenas de comparaciones donde la original hacía decenas de miles.

6. Concurrency avanzada · Map-reduce: dividir y combinar

Pista 1: `base, resto = divmod(10, 3)` da 3 y 1: un lote lleva 4 y los otros 3.

Pista 2: `total.get(clave, 0) + valor` acumula sin revisar si la clave existe.

```

1  def dividir_en_lotes(items, trabajadores):
2      """Lotes consecutivos y parejos (difieren como mucho en 1)."""
3      n = min(trabajadores, len(items))
4      if n == 0:
5          return []
6      base, resto = divmod(len(items), n)
7      lotes = []
8      inicio = 0
9      for i in range(n):
10         fin = inicio + base + (1 if i < resto else 0)
11         lotes.append(items[inicio:fin])
12         inicio = fin
13     return lotes
14
15
16 def combinar(resultados):
17     """Suma los conteos parciales por clave."""
18     total = {}
19     for parcial in resultados:
20         for clave, valor in parcial.items():
21             total[clave] = total.get(clave, 0) + valor
22     return total

```

Por qué funciona: `divmod` reparte el resto de a uno entre los primeros lotes, así la diferencia máxima es 1. `combinar` es el `reduce`: junta los resultados parciales de cada trabajador en uno solo.

7. Patrones de diseño pythónicos · Un bus de eventos (Observer)

Pista 1: `setdefault(evento, [])` crea la lista la primera vez.

Pista 2: Recorre una copia (`list(...)`) al emitir, por si algún suscriptor se desuscribe durante el aviso.

```

1 class EventBus:
2     """Publicador/suscriptor: bus.emitir("evento", datos) avisa a quienes se
   ↳ suscribieron."""
3
4     def __init__(self):
5         self._suscriptores = {}
6
7     def suscribir(self, evento, funcion):
8         self._suscriptores.setdefault(evento, []).append(funcion)
9         return funcion
10
11     def al(self, evento):
12         def decorador(funcion):
13             return self.suscribir(evento, funcion)
14         return decorador
15
16     def desuscribir(self, evento, funcion):
17         lista = self._suscriptores.get(evento, [])
18         if funcion in lista:
19             lista.remove(funcion)
20
21     def emitir(self, evento, *args, **kwargs):
22         return [f(*args, **kwargs) for f in list(self._suscriptores.get(evento,
   ↳ [])))]

```

Por qué funciona: El bus no sabe qué hacen sus suscriptores y ellos no saben quién emite: ese desacople permite agregar reacciones nuevas (un log, una notificación) sin tocar el código que produce el evento.

8. Testing avanzado · Tests con dobles de prueba

Pista 1: mensajes.append es un notificador falso perfecto: después revisas la lista.

Pista 2: Para el reintento, una tarea que cuenta sus llamadas en una lista y falla solo en la primera.

```

1  def test_tarea_exitosa_avisa_ok():
2      mensajes = []
3      resultado = ejecutar_tarea(lambda: 42, mensajes.append)
4      assert resultado is True
5      assert mensajes == ["ok: 42"]
6
7
8  def test_error_definitivo_avisa_y_devuelve_false():
9      mensajes = []
10
11     def siempre_falla():
12         raise ConnectionError("sin red")
13
14     assert ejecutar_tarea(siempre_falla, mensajes.append, reintentos=2) is False
15     assert mensajes == ["error: sin red"]
16
17
18  def test_reintento_exitoso():
19      mensajes, llamadas = [], []
20
21     def falla_una_vez():
22         llamadas.append(1)
23         if len(llamadas) == 1:
24             raise TimeoutError("lento")
25         return "listo"
26
27     assert ejecutar_tarea(falla_una_vez, mensajes.append, reintentos=1) is True
28     assert mensajes == ["ok: listo"]
29     assert len(llamadas) == 2

```

Por qué funciona: El notificador falso hace visible el efecto que se quiere verificar, sin enviar nada real. Comparar la lista completa de mensajes (y no solo que contenga algo) es lo que detecta el error sutil de avisar los fallos intermedios.

9. Rendimiento nativo · Pensar en colecciones completas

Pista 1: `zip(*matriz)` desempaqueta las filas como argumentos de `zip`.

Pista 2: Calcula `min` y `max` una sola vez, fuera de la comprensión.

```

1 def producto_punto(a, b):
2     """producto_punto([1, 2, 3], [4, 5, 6]) -> 32"""
3     if len(a) != len(b):
4         raise ValueError("Los vectores deben tener la misma longitud")
5     return sum(x * y for x, y in zip(a, b))
6
7
8 def transponer(matriz):
9     """[[1, 2, 3], [4, 5, 6]] -> [[1, 4], [2, 5], [3, 6]]"""
10    return [list(fila) for fila in zip(*matriz)]
11
12
13 def normalizar(valores):
14     """[10, 20, 30] -> [0.0, 0.5, 1.0]"""
15     if not valores:
16         return []
17     minimo, maximo = min(valores), max(valores)
18     if minimo == maximo:
19         return [0.0] * len(valores)
20     return [(v - minimo) / (maximo - minimo) for v in valores]

```

Por qué funciona: Cada función expresa una operación sobre colecciones enteras y delega el recorrido a funciones integradas en C. Es el mismo modo de pensar que exige NumPy: describir qué se calcula, no cómo se recorre.

10. Arquitectura y calidad · La raíz de composición

Pista 1: Una comprensión junta todos los nombres que no están en el registro.

Pista 2: App recibe funciones ya resueltas: no conoce el registro.

```

1 class App:
2     """Núcleo: aplica pasos a los datos. No sabe de dónde vienen."""
3
4     def __init__(self, pasos, reintentos=0):
5         self.pasos = list(pasos)
6         self.reintentos = reintentos
7
8     def procesar(self, dato):
9         for paso in self.pasos:
10             dato = paso(dato)
11         return dato
12
13     def procesar_lote(self, datos):
14         return [self.procesar(d) for d in datos]
15
16
17 def crear_app(config, registro):
18     """Raíz de composición: valida la configuración y arma la App."""
19     if "plugins" not in config:
20         raise ValueError("Falta la lista de plugins")
21     desconocidos = [n for n in config["plugins"] if n not in registro]
22     if desconocidos:
23         raise ValueError("Plugins desconocidos: " + ", ".join(desconocidos))
24     reintentos = config.get("reintentos", 0)
25     if isinstance(reintentos, bool) or not isinstance(reintentos, int) or not 0 <=
↵ reintentos <= 10:
26         raise ValueError("reintentos debe ser un entero entre 0 y 10")
27     return App([registro[n] for n in config["plugins"]], reintentos)

```

Por qué funciona: App es el núcleo: solo sabe aplicar pasos y se prueba sin configuración. crear_app es la raíz de composición: el único lugar que conoce el registro y el formato de la configuración, y que valida todo antes de construir.